

Instancia 1

Paso 1: Configuración del Sistema y MariaDB

1. **Instalar MariaDB:**
2. Bash

```
sudo dnf update -y
sudo dnf install mariadb105-server -y
sudo systemctl start mariadb
sudo systemctl enable mariadb
```

- 3.
- 4.
5. **Asegurar la instalación:**
6. Ejecuta `sudo mysql_secure_installation` y sigue los pasos (asigna una clave root).
7. **Configurar acceso remoto:**
8. Por defecto, MariaDB solo escucha en `localhost`. Edita el archivo de configuración:
9. `sudo nano /etc/my.cnf.d/mariadb-server.cnf` (o similar) y busca la línea `bind-address`.
Cámbiala a:
10. Ini, TOML

`bind-address = 0.0.0.0`

- 11.
12. Luego reinicia: `sudo systemctl restart mariadb`.
13. **Crear la Base de Datos y el Usuario:**
14. Entra a MariaDB (`sudo mysql -u root -p`) y ejecuta:
15. SQL

```
CREATE DATABASE farmacias_db;
USE farmacias_db;
```

```
CREATE TABLE farmacias (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(255),
  comuna VARCHAR(100),
  direccion VARCHAR(255),
  telefono VARCHAR(50),
  dia_semana VARCHAR(20), -- lunes, martes...
  apertura TIME,
  cierre TIME
);
```

-- Crear usuario para las otras instancias

```
CREATE USER 'api_user'@'%' IDENTIFIED BY 'tu_password_seguro';
GRANT ALL PRIVILEGES ON farmacias_db.* TO 'api_user'@'%';
FLUSH PRIVILEGES;
```

16.

17.

Paso 2: Crear la API Maestra (CRUD + Lógica de Negocio)

En esta instancia instalaremos **Python** y **Flask** para gestionar las peticiones de las instancias 2 y 3.

1. **Instalar dependencias:**

2. Bash

```
sudo dnf install python3-pip -y
pip3 install flask mysql-connector-python
```

3.

4.

5. **Crear el archivo `server_api.py`:**

6. Este código permite insertar datos y consultar si están abiertas según la hora actual.

Python

```
from flask import Flask, request, jsonify
import mysql.connector
from datetime import datetime
```

```
app = Flask(__name__)
```

```
# Configuración de conexión
```

```
db_config = {
    'host': 'localhost',
    'user': 'api_user',
    'password': 'tu_password_seguro',
    'database': 'farmacias_db'
}
```

```
def query_db(query, args=(), one=False):
    conn = mysql.connector.connect(**db_config)
    cursor = conn.cursor(dictionary=True)
    cursor.execute(query, args)
    rv = cursor.fetchall()
    conn.commit()
    cursor.close()
    conn.close()
    return (rv[0] if rv else None) if one else rv
```

```

@app.route('/farmacias', methods=['GET'])
def get_farmacias():
    comuna = request.args.get('comuna')
    solo_abiertas = request.args.get('abiertas') # True/False

    sql = "SELECT * FROM farmacias WHERE comuna = %s"
    params = [comuna]

    if solo_abiertas == 'true':
        ahora = datetime.now()
        # Mapeo de días al español según el dataset
        dias = ["lunes", "martes", "miercoles", "jueves", "viernes", "sabado", "domingo"]
        dia_nombre = dias[ahora.weekday()]
        hora_actual = ahora.strftime('%H:%M:%S')

        sql += " AND dia_semana = %s AND %s BETWEEN apertura AND cierre"
        params.extend([dia_nombre, hora_actual])

    results = query_db(sql, params)
    return jsonify(results)

@app.route('/farmacias', methods=['POST'])
def add_farmacia():
    datos = request.json
    sql = """INSERT INTO farmacias (nombre, comuna, direccion, telefono, dia_semana, apertura,
    cierre)
        VALUES (%s, %s, %s, %s, %s, %s, %s)"""
    valores = (datos['nombre'], datos['comuna'], datos['direccion'],
        datos['telefono'], datos['dia_semana'], datos['apertura'], datos['cierre'])

    query_db(sql, valores)
    return jsonify({"status": "success", "message": "Farmacia registrada"}), 201

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)

```

Paso 3: Configuración de Red (AWS Security Group)

Para que las Instancias 2 y 3 puedan hablar con la 1, debes ir a la consola de AWS y en el **Security Group de la Instancia 1** agregar estas reglas de entrada:

Tipo	Protocolo	Puerto	Origen	Descripción

Custom TCP	TCP	5000	IP_Instance_2 y 3	Para la API Flask
MYSQL/Aurora	TCP	3306	IP_Instance_2 y 3	Para conexión directa a DB (opcional)

Paso 4: Carga Inicial de Datos (Opcional pero recomendado)

Puedes crear un pequeño script en la Instancia 1 que lea la URL de `datos.gob.cl`, filtre las comunas que el grupo decidió y haga los primeros `INSERT` a la base de datos para que las otras instancias tengan qué consultar desde el primer minuto.

Ejecución:

Para dejar la API corriendo permanentemente en la Instancia 1, usa `nohup`:

Bash

```
nohup python3 server_api.py > api.log 2>&1 &
```

Instancia 2

Para la **Instancia 2**, el objetivo es empaquetar la lógica de consumo de la API en un contenedor Docker. Esto permitirá que la **Instancia 3** sea una réplica exacta simplemente ejecutando el mismo contenedor.

Aquí tienes el paso a paso para configurar el entorno de desarrollo del contenedor y desplegarlo.

1. Preparación de la Instancia 2

Conéctate por SSH e instala Docker:

Bash

```
sudo dnf update -y
sudo dnf install docker -y
sudo systemctl start docker
sudo systemctl enable docker
# Opcional: permite usar docker sin sudo
```

```
sudo usermod -aG docker $USER && newgrp docker
```

2. Creación del Entorno de la Aplicación

Crea una carpeta para el proyecto y entra en ella:

```
Bash
```

```
mkdir app-farmacia && cd app-farmacia
```

Debes crear **tres archivos** dentro de esta carpeta:

A. requirements.txt

Define las librerías necesarias.

```
Plaintext
```

```
flask  
requests
```

B. app.py (La lógica del Cliente)

Este script actúa como intermediario. Recibe peticiones de los usuarios y las redirige a la **Instancia 1**.

```
Python
```

```
from flask import Flask, request, jsonify  
import requests
```

```
app = Flask(__name__)
```

```
# REEMPLAZA CON LA IP PRIVADA O PÚBLICA DE LA INSTANCIA 1  
URL_MAESTRA = "http://172.31.xx.xx:5000/farmacias"
```

```
@app.route('/consultar', methods=['GET'])
```

```
def consultar():
```

```
    comuna = request.args.get('comuna')  
    abiertas = request.args.get('abiertas', 'false')
```

```
    # Redirige la consulta a la Instancia 1
```

```
    response = requests.get(URL_MAESTRA, params={'comuna': comuna, 'abiertas': abiertas})  
    return jsonify(response.json()), response.status_code
```

```
@app.route('/registrar', methods=['POST'])
```

```
def registrar():
```

```
    # Recibe datos del cliente y los manda a la Instancia 1
```

```
datos = request.json
response = requests.post(URL_MAESTRA, json=datos)
return jsonify(response.json()), response.status_code

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=80)
```

C. Dockerfile

La receta para construir la imagen.

Dockerfile

```
FROM python:3.9-slim
```

```
WORKDIR /app
```

```
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
COPY app.py .
```

```
# El contenedor escuchará en el puerto 80
```

```
EXPOSE 80
```

```
CMD ["python", "app.py"]
```

3. Construcción y Ejecución (Build & Run)

1. **Construir la imagen:**
2. Bash

```
docker build -t cliente-farmacia:v1 .
```

- 3.
- 4.
5. **Ejecutar el contenedor:**
6. Bash

```
docker run -d -p 80:80 --name app_cliente cliente-farmacia:v1
```

- 7.
- 8.

4. Replicación a la Instancia 3

Para que tu compañero en la **Instancia 3** tenga exactamente lo mismo sin volver a escribir el código, tienes dos opciones:

Opción Rápida (Docker Hub)

1. Crea una cuenta en [Docker Hub](#).
2. Desde la **Instancia 2**, sube tu imagen:
3. Bash

```
docker login
```

```
docker tag cliente-farmacia:v1 tu_usuario/cliente-farmacia:v1
```

```
docker push Sebamugi/cliente-farmacia:v1
```

- 4.
- 5.
6. En la **Instancia 3**, tu compañero solo necesita instalar Docker y correr:
7. Bash

```
docker run -d -p 80:80 tu_usuario/cliente-farmacia:v2
```

- 8.
- 9.

Opción Manual (Archivo comprimido)

Si no quieren usar Docker Hub, puedes exportar la imagen a un archivo:

1. **En Instancia 2:** `docker save cliente-farmacia:v1 > imagen_farmacia.tar`
2. Manda el archivo `.tar` a la Instancia 3 (vía `scp`).
3. **En Instancia 3:** `docker load < imagen_farmacia.tar` y luego `docker run`.

5. Prueba de funcionamiento

Desde cualquier computador con acceso a internet (o desde la misma terminal), prueba si la Instancia 2 responde correctamente consultando por una comuna que ya esté en la DB:

```
Bash
```

```
curl "http://IP_INSTANCIA_2/consultar?comuna=Coronel&abiertas=true"
```

Nota de seguridad: Asegúrate de que el **Security Group** de la Instancia 2 (y la 3) tenga abierto el **Puerto 80 (HTTP)** para que el mundo pueda consultar la API.

Instancia 3

1. Preparación de la Instancia 3

Al igual que en las anteriores, esta instancia debe tener el motor de Docker funcionando.

Bash

```
sudo dnf update -y
sudo dnf install docker -y
sudo systemctl start docker
sudo systemctl enable docker
sudo usermod -aG docker $USER && newgrp docker
```

2. Opción A: Despliegue mediante Docker Hub (Recomendado)

Si el compañero de la Instancia 2 subió la imagen a la nube, en la Instancia 3 solo ejecutas un comando:

1. **Descargar y correr el contenedor:**
2. Bash

```
# Reemplaza 'usuario' por el nombre de cuenta de tu compañero
docker run -d -p 80:80 --name cliente_replica Sebamugi/cliente-farmacia:v2
```

- 3.
4. *Docker detectará que no tienes la imagen localmente, la descargará de internet y la pondrá en marcha automáticamente.*

3. Opción B: Despliegue mediante Archivo (Manual)

Si prefieren transferir la imagen directamente entre servidores:

1. **Recibir el archivo:** Tu compañero de la Instancia 2 debe haberte enviado el archivo `imagen_farmacia.tar` (pueden usar `scp` o un pendrive virtual).
2. **Cargar la imagen en Docker:**
3. Bash

```
docker load < imagen_farmacia.tar
```

- 4.
- 5.

6. Ejecutar el contenedor:

7. Bash

```
docker run -d -p 80:80 --name cliente_replica cliente-farmacia:v1
```

8.

9.

4. Verificación del Sistema Grupal

Ahora que las 3 instancias están arriba, deben validar el flujo completo. Este es el test de oro para su trabajo:

Test 1: Registro desde Instancia 3

Desde la terminal de la Instancia 3, intenta registrar una farmacia de una comuna nueva (ej. Concepción):

Bash

```
curl -X POST http://localhost/registrar \
-H "Content-Type: application/json" \
-d '{
  "nombre": "Farmacia Universitaria",
  "comuna": "Concepcion",
  "direccion": "Barrio Universitario s/n",
  "telefono": "+5641222222",
  "dia_semana": "martes",
  "apertura": "08:00:00",
  "cierre": "21:00:00"
}'
```

Test 2: Consulta desde Instancia 2

Ahora, desde la Instancia 2 (o desde un navegador usando su IP pública), verifica si la farmacia que registró tu compañero en la Instancia 3 aparece:

Bash

```
curl "http://IP_INSTANCIA_2/consultar?comuna=Concepcion"
```

Resumen de la Red para el Informe

Para su documentación, la arquitectura quedó así:

Instancia	Rol	Software Clave	Puerto
Instancia 1	Servidor Maestro	MariaDB + Flask (API Central)	5000
Instancia 2	Nodo Cliente A	Docker (Contenedor Flask Cliente)	80
Instancia 3	Nodo Cliente B	Docker (Imagen replicada)	80

✓ Checklist Final:

- **Security Groups:** La Instancia 1 debe permitir tráfico en el puerto 5000 desde las IPs de la 2 y la 3.
- **Base de Datos:** La columna `comuna` en la Instancia 1 debe ser `VARCHAR` para permitir múltiples ciudades del dataset.
- **Persistencia:** Si reinician la Instancia 1, recuerden volver a ejecutar la API con `nohup`.

¿Tienen claridad sobre cómo automatizar la carga inicial de todas las comunas desde el link de Datos Gob, o lo harán manualmente?